Analysis of WHO Mortality Data for European Union Countries (2000–2020)

Dominika Nowak

Postgraduate Diploma in Big Data Analytics and Artificial Intelligence
Atlantic Technological University (ATU)

Email: L00203119@atu.ie

March 15, 2026

Abstract

This study analyzes WHO mortality data for European Union (EU) countries between 2000 and 2020 using the Databricks Free Edition distributed computing platform. The main goal was to explore trends in the leading causes of death in EU countries and to examine how these trends relate to healthcare expenditure indicators.

A big data analytics workflow was developed to clean, organize, and analyze large-scale mortality records. Two machine learning approaches were applied: Gradient Boosted Trees (GBT) for modeling and a combination of Principal Component Analysis (PCA) and K-means clustering to group countries with similar characteristics into risk-based categories. Both models were improved through hyperparameter tuning to achieve better performance and clearer interpretation.

The models were used to identify key mortality trends, simplify complex health expenditure indicators, and classify EU countries into different risk-level groups based on mortality patterns and healthcare spending relative to GDP. The results highlight the differences between countries and provide a data-driven basis to compare public health outcomes across the EU.

1 Introduction

The mortality dataset used in this study was obtained from the World Health Organization (WHO) Mortality Database, with a primary focus on the International Classification of Diseases, 10th Revision (ICD-10). The database covers the period from 1998 to 2024 and includes countries that report officially registered deaths coded according to ICD-10 standards. The dataset provides annual mortality counts by country, cause of death (ICD-10 code), age group, and sex.

The WHO Mortality Database is one of the most comprehensive global health data sources available. It forms part of the broader WHO Global Health Observatory, which provides extensive health-related data across multiple domains, including mortality statistics, public health indicators, health equity, environmental health, infectious diseases, and other major global health topics. This makes the dataset highly valuable for comparative public health research and policy analysis.

This study focuses specifically on the working-age population between 2000 and 2020 in the 27 member states of the European Union. The top three causes of mortality were identified during the exploratory data analysis (EDA) phase. The main objective is to analyze trends in these leading causes of death among the working-age population and to classify EU countries into different risk-level groupings. The analysis considers age-standardized rates (ASR), healthcare expenditure indicators, and broader mortality trends derived from large-scale data processing.

Industry literature emphasizes that data analysis alone is not enough to create meaningful impact without input from subject matter experts. This is especially true when studying mortality patterns, where knowledge in epidemiology and public health is important for proper interpretation [1]. As indicated in by domain experts: "As time advances, the mortality rates change for individuals in all risk categories. Modern advances in medicine and other areas produce a decrease in mortality rates as the future time variables increase"[2].

1.1 Aim

The aim of this project is to analyze WHO mortality data using the Databricks Free Edition distributed computing environment and generate meaningful insights into mortality trends, major causes of death, and demographic differences across countries, focusing on the working-age population.

1.2 Objectives

- Ingest and preprocess WHO mortality data within a scalable Databricks distributed computing environment to ensure efficient handling of large-scale datasets.
- Clean, validate, and standardize key variables, including country, year, sex, age group, and cause of death, to ensure data consistency and analytical reliability.
- Conduct comprehensive exploratory data analysis (EDA) to examine mortality distributions, patterns, and summary statistics.
- Identify and evaluate significant mortality trends over time across European Union member states.
- Compare mortality patterns by sex, age group

(15–69 years), and major cause-of-death categories.

- Assess the effectiveness of Databricks as a platform for large-scale public health data analytics and machine learning, while managing the limitations of the Free Edition to successfully deliver the results of this project

2 Dataset Description

2.1 Data Source

The primary data source for this study was the WHO Mortality Dataset, provided in CSV format and structured according to the ICD-10 classification system. Specifically, six files labeled Mortality, ICD-10 Part 1–6 were used. These files were combined and integrated within the Databricks SQL environment to create a unified table named who mortality under the who mortality schema. This consolidation enabled efficient querying, aggregation, and large-scale analysis of mortality records across countries, years, age groups, and causes of death.

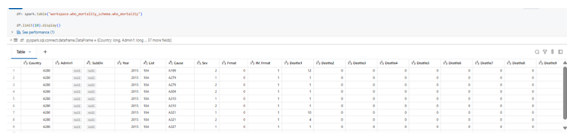

Figure 1: Databricks SQL table for WHO mortality dataset integration.

It is important to note that the Mortality, ICD-9 dataset was also used to address gaps for countries that adopted ICD-10 cause coding later than most EU member states. For example, Ireland began reporting mortality data under ICD-10 in 2007. To ensure continuity and comparability across the full study period, selected records from ICD-9 were incorporated where necessary. A mapping process was applied to align ICD-9 codes with ICD-10 classifications for the main causes of death, including cancer, allowing for a more consistent analysis[3].

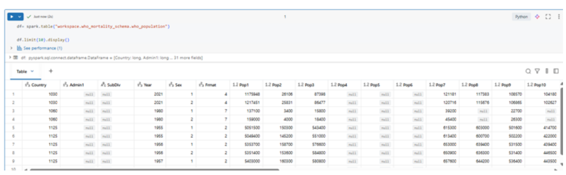

Figure 2: Databricks SQL table for WHO population

The WHO country codes CSV file allowing identification of the EU countries. The WHO

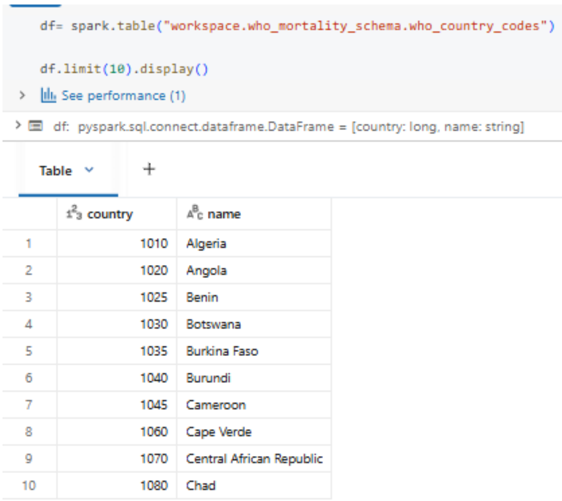

Figure 3: Databricks SQL table for WHO country mapping

documentation for ICD-10 was not sufficient source for cause identification [4]. However one of the official resources accessible online for medical professionals allowed detailed classification. [5]. The data collection follows the International Classification of Diseases (ICD) system, which has been revised roughly every decade since 1900 and is overseen by WHO. Version 11 was released in 2018. The classification system is remarkably detailed, though this complexity raises questions about implementation consistency across different healthcare systems.[6] The Healthcare expenditure

Figure 4: Official ICD-10 Resource for Detailed Verification of Codes

dataset was extracted from WHO Global Health Expenditure Database.The annual indicators used during modelling process where the data columns corresponding to features:

- CHE GDP: Current Health Expenditure as a percentage of GDP.
- CHE PC USD: Current Health Expenditure per capita (US dollars).
- CHE NCU: Current Health Expenditure in million national currency units (NCU).

```
df= spark.table("workspace.who_mortality_schema.health_expenditure")

df.limit(10).display()
```

See performance (1)

df: pyspark.sql.connect.dataframe.DataFrame = [location: string, code: string ... 6 more fields]

Table

	location	code	region	income	year	che_gdp	che_pc_usd	che
1	Czech Republic	AFG	EMR	Low	2002	9.4433908500000001	16.70697403000000...	16880.626953129999000...
2	Czech Republic	AFG	EMR	Low	2003	8.9412584300000013	17.74602509000000...	19671.931640629999000...
3	Czech Republic	AFG	EMR	Low	2004	9.8084735900000002	21.42300415000000...	24149.441406250000000...
4	Czech Republic	AFG	EMR	Low	2005	9.9482889200000013	25.11388779000000...	30334.919921879995000...

Figure 5: Current Health Expenditure tbl

2.2 Data Characteristics

Health outcomes vary greatly by age, which makes age structure an important factor in mortality analysis. If differences in age distribution are not considered, comparisons between countries can be misleading. For example, a country with more elderly people may have a higher crude mortality rate even if the actual risk of death at each age is the same. This is why age-standardized rates (ASR) are essential when comparing populations[7].
Mortality data also has several limitations. The WHO has acknowledged significant gaps in data availability. "The WHO explicitly states, though, that disaggregation is not possible for all indicators; meaning that the national statistics which serve as the sources are deficient. Hence, the WHO reports are incomplete"[8]. Errors in death certificates, misclassification of causes of death, inconsistencies between countries, and gaps in reporting can affect data accuracy[9]. Differences in data collection methods and self-reported health measures can further reduce comparability. Despite these challenges, the WHO Mortality Database remains an important and valuable source for international health research, as long as the data is interpreted carefully and with awareness of its limitations. The WHO mortality dataset has several big data characteristics:

3 Methodology

3.1 Analytical Environment

Databricks free version served as the analytical platform for this study, providing storage and management of the required data files as well as an environment for developing, training, and evaluating the machine learning models. Databricks is a unified data analytics platform built on Apache Spark that combines data engineering, data science, and machine learning capabilities in a single cloud-based environment. Modern Databricks deployments leverage the Photon Engine for high-performance processing and provide comprehensive data governance through Unity Catalog[10].

3.2 Workflow Stages

Using the Big Data pipeline, Databricks supported several key stages of the project indicated below.

1. Data Ingestion and Storage

All the CSV dataset files were uploaded into the Databricks environment and transformed into structured tables within the WHO mortality schema. By converting the raw CSV files into Spark-managed tables, the data became fully integrated into the distributed computing framework. The schema-based organization of data improved performance for large-scale aggregations, joins, and analytical transformations performed in python notebooks during data processing and modelling phase as well as supported reproducibility and consistent access to datasets throughout the analytical workflow

2. Data Cleaning, Transformation and Exploratory Data Analysis (EDA)

Data was cleaned, filtered, aggregated, and transformed using Spark DataFrames. This included filtering the relevant years and countries, grouping the data by country and cause of death, and ensuring that missing or incorrect values were handled in proper manner. Distributed operations ensured that computations were performed efficiently. An initial data exploration was conducted to understand the structure and quality of all the datasets, WHO mortality dataset especially. Basic dataset statistics were generated to examine the data patterns and identify missing values or inconsistencies. This step ensured the data were suitable for further analysis and modeling. Actions performed after initial data exploration:

- Identified the relevant age groups representing the working-age population within the WHO mortality, population, and European Standard Population datasets for subsequent analysis.
- Filtered the dataset to include only working-age population groups, selected years:2000-2020, and European Union member countries.
- Conducted an initial review of ICD-10 codes to understand the guidelines for identifying causes

- Resolve inconsistencies in country naming within the CSV files (for example, variations in the naming of the Czech Republic) to ensure data accuracy and consistency across the dataset.
- Identified and supplemented missing records in WHO ICD-10 mortality and WHO population data, incorporating ICD-9 data for countries that delayed transitioning to the new classification system. Mapped ICD-9 primary causes to their corresponding ICD-10 codes for use in data analysis and modeling.

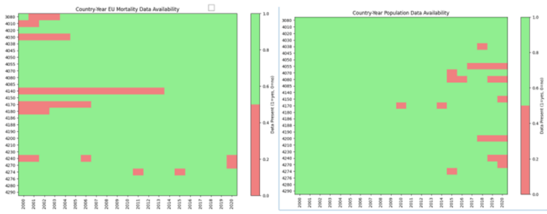

Figure 6: Who Missing Records Heatmap

- Identified persistent missing records in EU datasets following interpolation. Interpolation is a powerful technique for estimating missing values in time series data by using mathematical methods to fill gaps based on surrounding data points. Used in project, Linear interpolation is the most straightforward approach, creating a straight line between known data points to estimate missing values. As explained in Python Data Cleaning and Preparation Best Practices 1, "interpolation allows us to estimate missing data points by leveraging the existing values in the dataset, providing a more nuanced approach that can capture trends and patterns over time"[11].
- Remaining missing values were updated using the last available previous record. This approach was considered appropriate due to the relatively stable and consistent trends observed within the working-age population groups.

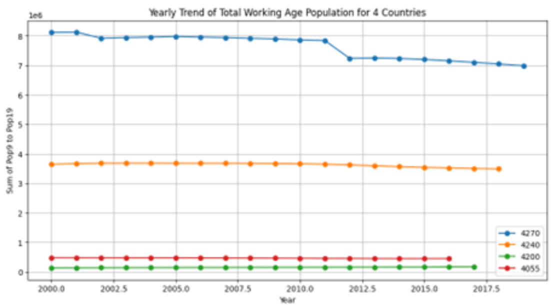

Figure 7: Who Persisting Missing Records Trends

- Identified the top three causes of mortality across the EU and within each country using both raw death counts and ASR data. The results were highly consistent, highlighting cancer, heart, and liver related causes as the leading contributors, supporting their use in future prediction and clustering analyses.

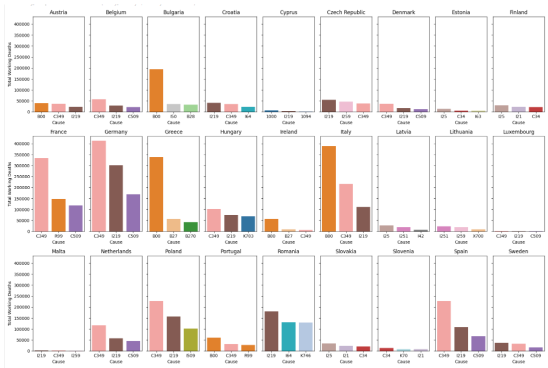

Figure 8: Top 3 Mortality Causes Per EU Country

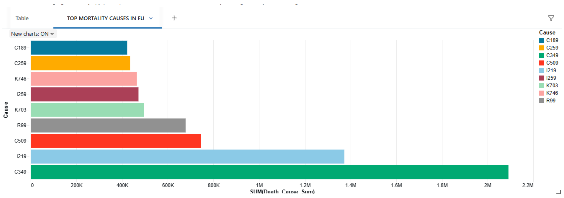

Figure 9: Top Mortality Causes in EU

- Extracted the relevant ICD-10 code ranges corresponding to the top three causes of mortality for use in the analysis.

```
liver_codes_range = ["K70","K77"]
liver_codes= [f"K{str(i)}" for i in range(70, 78)]
print(liver_codes)

#prefix single digits with 0
cancer_codes_range =["C00","C97"]
cancer_codes = [f"C{i:02d}" for i in range(0, 98)]
print(cancer_codes)

heart_codes_range = ["I20","I25"]
heart_codes = [f"I{str(i)}" for i in range(20, 26)]
print(heart_codes)
```

Figure 10: ICD-10 Codes For Top Causes

- Performed data summarization and aggregation from multiple perspectives, analyzing various features and dimensions to gain deeper insights and identify potential data quality issues across all datasets.
- Finalized this stage by saving the prepared dataset for further processing and analysis.

```python
# Save DataFrame as Delta Table with valid column names, overwrite if exists
from pyspark.sql import functions as F

def fix_col_names(df):
    for c in df.columns:
        if (' ' in c) or any(x in c for x in ",;{}()\n\t="):
            df = df.withColumnRenamed(c, c.replace(' ', '_').replace('/', '_'))
    return df

df_clean = fix_col_names(df_EU_ASR)
df_clean.write.format("delta").mode("overwrite").saveAsTable("who_mortality_schema.eu_mortality")
```

Figure 11: Database Table For Extracted Features

3. Feature Engineering

Additional variables were created to support the analysis., including age-standardized mortality rates (ASR), lagged indicators, and aggregated working-age mortality measures, were computed using distributed Spark functions and window operations. Calculated ASR data using year, sex, and country specific mortality and population data, along with ESP weights, for each working-age population group (groups 9–19). WHO Population files for working age ranges corresponding to mortality data were used for the purpose of estimating ASR for each European country. The Age-Standardized Mortality Rate (ASR) for each working age group was used to adjusts mortality rates to a standard age distribution to remove the effect of differences in population age structure between countries or over time. Example: For the age group i

$$r_i = \frac{D_i}{P_i} \times 100{,}000$$

$$r_i = \frac{D_i}{P_i}$$

where:

- r_i is the age-specific mortality rate for age group i,
- D_i is the number of deaths in age group i,
- P_i is the population in age group i.

```python
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from pyspark.sql.functions import col
from functools import reduce

# Pair up death and population columns by age group suffix
death_pop_pairs = [(d, p) for d, p in zip(working_mort_age_cols, working_pop_cols) if d[6:] == p[3:]]
print(death_pop_pairs)

df_EU_ASR = df_EU_mortality_population_joined
for d_col, p_col in death_pop_pairs:
    age_group = d_col[6:]
    esp_weight =  dict_pop_ESP.get(age_group, 0)
    asr_col = f"ASR{age_group}"
    df_EU_ASR = df_EU_ASR.withColumn(
        asr_col,
       (((col(d_col) / col(p_col)) *100000) * esp_weight/100000)
    )
```

Figure 12: Working Population ASR Calculation

4. Application Gradient Boosted Trees(GBT)

The objective of modeling stage is to build a forecasting model that predicts future trends in the three main causes of mortality among the working-age population in the European Union using historical WHO mortality and calculated ASR data. Gradient Boosted Trees (GBT) was selected for this time-series analysis because it is well suited to capturing nonlinear relationships and complex interactions. The dataset is panel time-series data, and GBT performs efficiently with multiple predictors and structured tabular formats. Unlike classical models such as linear regression or ARIMA, which assume linearity and certain levels of normality, mortality data often violates these assumptions and requires additional feature engineering (e.g., lag variables). GBT does not rely on strict statistical assumptions and can naturally model complex patterns. Additionally, GBT provides strong predictive accuracy, is resistant to overfitting when properly tuned, and performs well on medium-to-large structured datasets. Given the limitations and inconsistencies within mortality data, tree-based methods offer a more robust and flexible modeling approach.

Dataset Feature Adjustment for GBT

The following steps were carried out during this stage tor data features:

- Grouped the dataset according to the three primary causes of mortality: cancer, heart, and liver.
- Selected and extracted key domain-relevant features for modeling purposes.
- Applied lag variables to capture temporal patterns and support time-series forecasting. Tree-based models like Gradient Boosted Trees do not inherently understand time order. By adding lag variables, we explicitly provide time structure to the model. Lagged variables allow models to learn how current values relate to past observations.When a time series is correlated with a lagged version of itself, we say it's autocorrelated.This autocorrelation contains valuable information for forecasting future values [12].

Data Splitting and Model Training

During Data Splitting and Model Training stage, the dataset was split into training and testing sets using a chronological approach to preserve the time-series structure. Data from 2000 to 2017 was used as the training set, while data from 2018 to 2020 was reserved for testing. This time-based split ensures that the model is trained only on

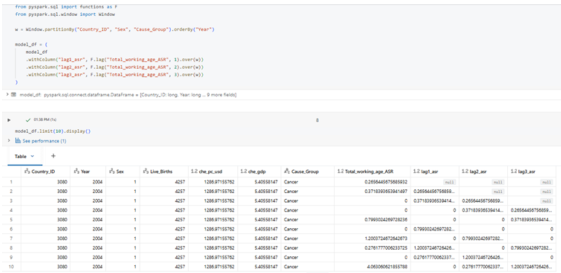

Figure 13: GBT Lagging Python Screenshot

past information and evaluated on future, unseen data, which reflects a realistic forecasting scenario. Using a chronological split also prevents data leakage and allows for a more reliable assessment of the model's predictive performance.

The StringIndexer was applied to encode categorical variables into numerical indices, enabling them to be used as input features in the machine learning model.

Tune Hyperparamters Used a VectorAssembler to combine the selected indexed categorical variables, time indicator, health expenditure features, and lag-based ASR variables into a single feature vector for model input. These input columns were evaluated for their contribution to model performance, while structuring them through the assembler also helped manage feature handling efficiently and avoid potential Databricks fee edition cache sizing issues.

```
assembler = VectorAssembler(
    inputCols=[
        "country_idx",
        "sex_idx",
        "cause_idx",
        "year_index",
        "che_pc_usd",
        "che_gdp",
        "lag1_asr",
        "lag2_asr",
        "lag3_asr",
        "rolling3_asr"
    ],
    outputCol="features",
    handleInvalid="skip"
)
```

```
#Train the model

gbt = GBTRegressor(
    featuresCol="features",
    labelCol="Total_working_age_ASR",
    predictionCol="prediction",
    maxIter=30,
    maxDepth=5,
    stepSize=0.1,
    subsamplingRate=0.8,
    seed=42
)
pipeline = Pipeline(stages=[
    country_indexer,
    sex_indexer,
    cause_indexer,
    assembler,
    gbt
])

model = pipeline.fit(train_df)
```

Figure 14: GBT Settings Screenshot

The optimal approach involved selecting features that improved model performance while tuning the hyperparameters—setting the maximum iterations to 30 and the maximum depth to 5—to accommodate the resource constraints of the Databricks Community Edition. The reoccuring issue: Your fitted model is about 365 MB, and Databricks Serverless / Spark Connect has a hard limit of 256 MB. So this is a model complexity / representation problem, not a data-quality problem.

Model Evaluation

Model performance was evaluated using three regression metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2. These metrics assessed prediction accuracy, average error magnitude, and the proportion of variance explained by the model, respectively. Removing live births and population features did not reduce model performance, indicating that these variables were not contributing meaningful predictive power to the model. The performance metrics are almost identical. Their removal simplifies the model without sacrificing performance, which is beneficial given computational constraints in the Databricks Community Edition

```
RMSE: 2.130666086310785     RMSE: 2.1318280938792875
MAE: 0.7518154826658191     MAE: 0.74514475436607
R2: 0.07234693632152867     R2: 0.07133482670240776
```

Figure 15: GBT Evaluation Metrix Results

Evaluation of GBT Forecasting Performance

The Gradient Boosted Trees (GBT) model does not show strong explanatory or forecasting performance in its current setup. Although it produces reasonable predictions, it does not fully capture the deeper structural factors influencing mortality trends. This does not mean the model is incorrect; rather, it suggests that age-standardized rates (ASR) may be more difficult to predict with the current number of features and a 20-year time span. The model could likely be improved by adding more relevant variables and extending the lag period to include a longer historical timeframe.

5. Application of Principal Component Analysis (PCA) and K-Means Clustering

PCA (Principal Component Analysis) is commonly used with clustering algorithms for several compelling reasons that improve both the effectiveness and efficiency of the clustering process.PCA allows you to reduce complex datasets while preserving the most important information. There's a deeper mathematical relationship between PCA and clustering that justifies their combination. "It turns out that principal components are actually the continuous solution of the cluster membership indicators in the k-means clustering method, i.e. the PCA dimension reduction automatically performs data clustering according to the k-means objective function. This provides an important justification of PCA-based data reduction."[13] PCA addresses this "curse of dimensionality" by compressing high-dimensional data into a lower-dimensional representation while preserving the global structure[14]. This reduction brings multiple benefits: decreases memory usage - reducing data from four to two dimensions halves the storage requirements and it provides faster computations [15].

Feature Scaling for PCA K-Clustering

At this stage, the data was aggregated at the country level by calculating the average working-age ASR for each of the three main causes of mortality (cancer, heart, and liver). The dataset was then reshaped using a pivot operation so that each country had one row with separate columns representing the average ASR for each cause. Missing values were removed to ensure completeness. This structured country-level dataset was prepared for Principal Component Analysis (PCA) to reduce dimensionality and identify dominant mortality patterns, followed by clustering to group countries with similar mortality profiles.
Selected Cancer, Heart, and Liver ASR values as input features, combined them into a single feature vector using VectorAssembler, and applied StandardScaler to standardize the data (mean-centered and scaled by standard deviation) in preparation for PCA and clustering. The Vector Assembler combined the three separate columns (Cancer, Heart, Liver) into a single numerical feature vector called "features". This format is required for Spark ML algorithms such as PCA and K-means. If scaling is not applied:

- Variables with larger magnitude dominate PCA and clustering
- Clusters may reflect scale differences rather than real mortality structure

Standardization ensures that each cause contributes equally to pattern detection. After scaling each country is represented by a normalized 3-dimensional vector.

```
01:01 PM (5s)
from pyspark.ml.feature import VectorAssembler, StandardScaler

feature_cols = ["Cancer", "Heart", "Liver"]

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features"
)

df_vec = assembler.transform(df_EU_country)

scaler = StandardScaler(
    inputCol="features",
    outputCol="scaled_features",
    withMean=True,
    withStd=True
)

scaler_model = scaler.fit(df_vec)
df_scaled = scaler_model.transform(df_vec)
```

Figure 16: Standard Scaler

PCA K-Cluster Implementation

Principal Component Analysis (PCA) was implemented using the PySpark ML library to reduce the dimensionality of the scaled feature set. The model was configured with two principal components (k=2), transforming the original feature space into a new set of orthogonal components stored in the column pca features. The explained variance output indicates how much of the total variance in the data is captured by each principal component. The first principal component explains approximately 81.13% of the variance, while the second explains about 11.26%. Together, the two components account for roughly 92% of the total variance, suggesting that most of the information contained in the original features can be effectively represented in a two-dimensional space. This confirms that dimensionality reduction is appropriate while retaining the majority of the dataset's structural information. K-Means clustering was implemented using the PySpark ML library to group observations based on the reduced feature space obtained from PCA. The model was configured with three clusters (k=3) and applied to the pca features column. After fitting the model, cluster assignments were generated and stored in the cluster column of the transformed dataset. This process enabled the grouping of countries into three distinct clusters based on similarities in their underlying mortality-related features.

PCA K-Cluster Model Evaluation

The quality of the K-Means clustering model was evaluated using the Silhouette score through PySpark's ClusteringEvaluator. The elbow method is a widely-used heuristic technique for determining

```python
from pyspark.ml.feature import PCA

pca = PCA(
    k=2,
    inputCol="scaled_features",
    outputCol="pca_features"
)

pca_model = pca.fit(df_scaled)
df_pca = pca_model.transform(df_scaled)

print("Explained variance:")
print(pca_model.explainedVariance)
```

```
df_pca: pyspark.sql.connect.dataframe.DataFrame
Explained variance:
[0.811314400600946,0.11262405787286757]
```

```python
from pyspark.ml.clustering import KMeans

# KMeans
kmeans = KMeans(k=3, featuresCol="pca_features", predictionCol="cluster", seed=42)
kmeans_model = kmeans.fit(df_pca)
df_clustered = kmeans_model.transform(df_pca)
```

Figure 17: PCA and K-Cluster Implementation

the optimal number of clusters (k) in K-means clustering algorithms. This evaluation method helps solve one of the fundamental challenges in unsupervised learning: deciding how many clusters best represent your data structure.[16] The Silhouette score measures how well each observation fits within its assigned cluster compared to other clusters, with values ranging from -1 to 1. In this case, the model achieved a Silhouette score of approximately 0.67, which indicates a reasonably strong clustering structure. A score above 0.5 generally suggests well-separated and meaningful clusters, implying that the three-cluster solution provides a good balance between cohesion within clusters and separation between clusters. This result supports the suitability of the chosen k=3 configuration for grouping the mortality-related features.

```python
from pyspark.ml.evaluation import ClusteringEvaluator

evaluator = ClusteringEvaluator(
    featuresCol="pca_features",
    predictionCol="cluster"
)

print("Silhouette score:", evaluator.evaluate(df_clustered))
```

```
Silhouette score: 0.6684928765075256
```

Figure 18: PCA and K-Cluster Evaluation

Evaluation of PCA K-Cluster Risk Grouping Performance

The clustering results indicate that countries within each cluster share similar characteristics, while the clusters themselves are clearly separated from one another. This suggests that the grouping structure is meaningful rather than random. In the context of the mortality study, the PCA results showed that the first principal component explains approximately 81% of the total variance, representing overall mortality intensity. Combined with a Silhouette score of approximately 0.67 from the K-means model, this provides strong evidence that EU countries naturally group into distinct and well-separated mortality intensity levels, such as low, medium, and high mortality clusters. Overall, the clustering structure can be considered statistically robust and analytically meaningful. The visualisation results support established patterns, indicating that Eastern European countries tend to fall within the highest risk category, while Mediterranean countries are generally grouped within the lower risk category.

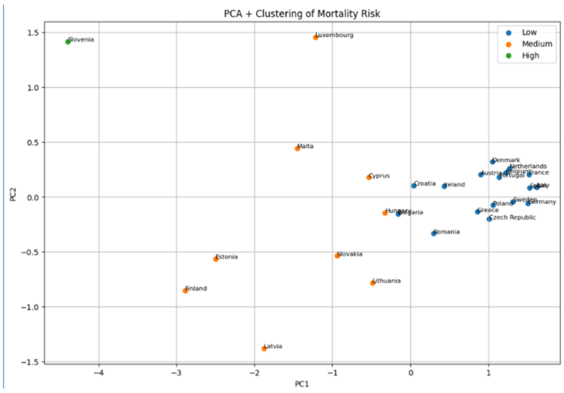

Figure 19: PCA and K-means clustering visualization of EU mortality risk

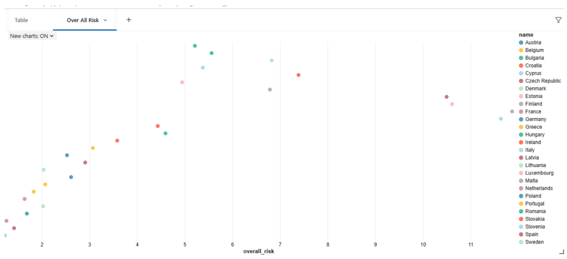

Figure 20: Over All Mortality Risk

4 Future Improvements

Future improvements for GBT model should focus on improving the quality of the predictive signal rather than only tuning model parameters, especially given the current low R^2 (0.07). Strengthening the lag structure and adding a clear time trend variable could help the model better capture long-term mortality patterns. Expanding the feature set with more structural predictors—such as alcohol consumption, obesity rates, unemployment,air pollution and urbanization may significantly improve model outputs. Hyperparameter optimization would unlikely lead to major improvements on its own. Separating the modeling of each mortality cause could also enhance accuracy by capturing cause-specific dynamics. Finally, replacing or complementing the current GBT model with alternative approaches, such as Random Forest, XGBoost, or time-series models, may improve forecasting performance.

5 Conclusion

This report analyzed WHO mortality data using Databricks to explore large-scale mortality patterns across time, geography, sex, age, and cause of death. The project shows that big data platforms are well suited to handling complex public health datasets and can generate meaningful insights for health monitoring and policy support.The models applied. This study applied two machine learning approaches—Gradient Boosted Trees (GBT) and a combination of Principal Component Analysis (PCA) with K-Means clustering—to analyze WHO mortality data for European Union countries between 2000 and 2020. The GBT model was used to explore the predictive relationship between structural health and economic indicators and age-standardized mortality rates. While the model produced reasonable predictions, its explanatory power remained limited, suggesting that mortality dynamics are influenced by complex structural factors that require richer feature sets and longer time horizons to model effectively. //In contrast, the PCA and K-Means clustering approach provided strong insights into the underlying structure of the data. PCA successfully reduced dimensionality while retaining the majority of the variance, and clustering results revealed clear and well-separated groupings of EU countries based on mortality intensity levels. The Silhouette score indicated that the clustering structure was statistically meaningful rather than random.
//Overall, the combination of predictive modeling and unsupervised learning offered complementary perspectives: GBT assessed forecasting capability, while PCA and clustering revealed structural patterns across countries. Together, these approaches demonstrate the value of big data analytics and machine learning techniques in understanding large-scale public health trends and supporting comparative mortality analysis within the European Union.

References

[1] J. M. L. Ferres, W. B. Weeks, and B. Smith, *AI for Good*. John Wiley & Sons, 2024.

[2] C. Skiadas and J. R. Bozeman, *Data Analysis and Applications 1*. John Wiley & Sons, 2019.

[3] National Bureau of Economic Research, "Icd-9-cm to and from icd-10-cm and icd-10-pcs crosswalk or general equivalence mappings," https://www.nber.org/research/data/icd-9-cm-and-icd-10-cm-and-icd-10-pcs-crosswalk-or-general-equivalence-mappings, 2026, accessed: Jan. 28, 2026.

[4] World Health Organization, "Icd–10 version 2019 online browsing platform," 2019, accessed: Feb. 1, 2026. [Online]. Available: https://icd.who.int/browse10/2019/en

[5] German Federal Institute for Drugs and Medical Devices (BfArM), "Icd-10-gm 2025 online code search," 2025, accessed: Feb. 1, 2026. [Online]. Available: https://klassifikationen.bfarm.de/icd-10-gm/kode-suche/htmlgm2025/index.htm

[6] B. B. Hughes, *International Futures*. Routledge, 2023.

[7] P. Muennig and M. Bounthavong, *Cost-Effectiveness Analysis in Health: A Practical Approach*, 2nd ed. Jossey-Bass, 2016.

[8] R. Bardy, *Public Goods and Commons*. Routledge, 2019.

[9] J. J. Berman, *Principles and Practice of Big Data*, 2nd ed. Academic Press, 2018.

[10] W. Girtin, *Building Modern Data Applications Using Databricks Lakehouse*. O'Reilly Media, 2023.

[11] M. Zervou, *Python Data Cleaning and Preparation Best Practices*. Packt Publishing, 2023.

[12] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2022.

[13] N. Benvenuto, G. Cherubini, and S. Tomasin, *Algorithms for Communications Systems and Their Applications*, 2nd ed. Wiley, 2021.

[14] J. Alammar and M. Grootendorst, *Hands-On Large Language Models*. O'Reilly Media, 2024.

[15] L. Apeltsin, *Data Science Bookcamp*. Manning Publications, 2021.

[16] P. Dangeti, *Statistics for Machine Learning*. Packt Publishing, 2017.